

2.4 Loop Output

Sue can't seem to find the bug in her assignment. The code looks right and it compiles, but the loop keeps giving her different output than she expects. How can she ever hope to find the bug if everything executes so quickly? If only there was a way to step through the code one line at a time to see what each statement is doing...

Objectives

By the end of this class, you will be able to:

- Predict the output of a given block of code using the desk check technique.
- Recognize common pitfalls associated with loops.

Prerequisites

Before reading this section, please make sure you are able to:

- Demonstrate the correct syntax for a WHILE, DO-WHILE, and FOR loop (Chapter 2.3).
- Create a loop to solve a simple problem (Chapter 2.3).
- Use `#ifdef` to create debug code in order to test a function (Chapter 2.1).

Desk Check

Please watch the following videos:

- [Predicting Output](#): The purpose of this video is to illustrate the importance of being able to predict the output of code by inspection.
- [Levels of Understanding](#): The purpose of this video is to illustrate different levels of understanding of an algorithm and the benefits of working at each level. There are three levels:
 1. **Concrete**: A low level understanding of the specific details of what the code is doing. It consists of a description of what happens at every step of the program execution.
 2. **Abstract**: Describes what parts of a program do, and how they relate to the larger whole.
 3. **Conceptual**: A high level comprehension that enables the programmer to explain in simple English what a program does.
- [Dataflow](#): The purpose of this video is to illustrate that tracking dataflow is the most effective way to predict the output of a program.
- [Desk Check Steps](#): The process of desk checking a program, from start to finish. It will describe how desk checking is a form of dataflow measurement, how the complete state of the program is captured at each level, and how every aspect of the algorithm is captured in the end. This chapter will describe the steps of performing a desk check, including line numbering, variable enumeration, and finally building the desk check table.
- [Desk Check Table](#): The purpose of this video is to illustrate how to build and interpret a desk check table on a single-function problem.
- [Desk Check with Functions](#): The purpose of this video is to illustrate how desk checking works across multiple functions.
- [Online Desk Check](#): The purpose of this video is to illustrate how to desk check existing code.



Example 2.4 – Expression Desk Check

Desk check the following code:

```
{
    // get paid!
    double income = 125.37;

    // remove tithing, you keep 90%
    income *= 0.9;

    // don't forget half goes to savings
    income /= 2.0;

    // a man has got to eat
    income -= 15.50;

    // display the results
    cout.setf(ios::fixed);
    cout.setf(ios::showpoint);
    cout.precision(2);
    cout << "$" << income << endl; }
```

The first step is to number the lines of code. These line numbers will then correspond to the rows in the Desk Check table. For convenience, the lines numbers are displayed in the line comments.

```
{
    double income = 125.37;           // (1) extra code removed for brevity
    income *= 0.9;                     // (2)
    income /= 2.0;                     // (3)
    income -= 15.50;                   // (4)
    cout << income << endl;           // (5)
}
```

Next, a table will be created to reflect the state of the variables at various stages of execution.

Line	income	output
1	125.37	
2	112.833	
3	56.4165	
4	40.9165	
5	40.9165	\$40.92

Notice how we are able to track each step of execution of the code with the Desk Check table. Even though income changed many times in a few lines of code, we can always see the value at a given moment in time.

Example 2.4 – Conditional Desk Check

Desk check the following code:

```
int computeTax(double income)
{
    // 10%, 15%, 25%, 28%, and 33% brackets here.
    if (income < 0.00)
        return 0;
    else if (income <= 15100.00)
        return 10;
    else if (income <= 61300.00)
        return 15;
    else if (income <= 123700.00)
        return 25;
    else if (income <= 188450.00)
        return 28;
    else if (income <= 336550.00)
        return 33;

    return 35;
}
```

The first step is to number the lines of code. These line numbers will then correspond to the rows in the Desk Check table. For convenience, the lines numbers are displayed in the line comments.

```
int computeTax(double income)
{
    if (income < 0.00)                // (1)
        return 0;                    // (2)
    else if (income <= 15100.00)      // (3)
        return 10;                   // (4)
    else if (income <= 61300.00)      // (5)
        return 15;                   // (6)
    else if (income <= 123700.00)     // (7)
        return 25;                   // (8)
    else if (income <= 188450.00)     // (9)
        return 28;                   // (10)
    else if (income <= 336550.00)     // (11)
        return 33;

    return 35;                        // (12)
}
```

Next, a table will be created to reflect the state of the variables at various stages of execution. We will start with income set to \$50,000.00

Line	income	return
1	50000.0	
3	50000.0	
5	50000.0	
6	50000.0	15

Note how line (2) is not executed because the Boolean expression in line (1) evaluated to `false`. This means that we move to line (3). Since it too evaluated to `false`, we move on to line (5). Now since $(50000.0 \leq 61300.00)$ evaluates to `true`, we move on to line (6). Line (6) is a return statement, meaning it is the last line of the function we execute.

It is important to realize that line (2), (4), (7), (8), (9), (10), (11), and (12) are never executed.

Example 2.4 – Loop Desk Check

Desk check the following code:

```
{
    int i;
    int j = 0;

    for (i = 1; i < 10; i *= 2)
        j += i;

    cout << j << endl;
}
```

The first step is to number the lines of code. These line numbers will then correspond to the rows in the Desk Check table. For convenience, the lines numbers are displayed in the line comments.

```
{
    int i;
    int j = 0;                // (1) along with the preceding line

    for (i = 1; i < 10; i *= 2) // (2)
        j += i;               // (3)

    cout << j << endl;        // (4)
}
```

Next, a table will be created to reflect the state of the variables at various stages of execution.

Line	i	j
1	?	0
2	1	0
3	1	1
2	2	1
3	2	3
2	4	3
3	4	7
2	8	7
3	8	15
2	16	15
4	16	15

Observe how the desk-check is a record of the execution of the loop. You can always “look back in time” to see exactly what was happening at a given stage in execution. For example, the third execution of the loop (highlighted) occurred with *i* set to 4 and *j* set to 3. Since *i* < 10 evaluated to true (because 4 is less than 10), the loop continued on to the body (step 3). From here, *i* remained unchanged, but *j* increased its value by 4. You can always read the current values of all the variables off the desk check table. As expected, the value of *j* jumped from 3 to 7 on this line of code

Notice how the FOR loop has three components: the Initialization, the Boolean expression, and the Increment. As a challenge, create a desk check where step 2 is split into these three components:

2a: Initialization

2b: Boolean expression

2c: Increment

Example 2.4 – Online Desk Check

Problem

Modify the following code to perform an online desk check.

```
{
    int i;
    int j = 0;

    for (i = 1; i < 10; i *= 2)
        j += i;

    cout << j << endl;
}
```

The first step is to insert a COUT statement labeling the variables that will be displayed in the various columns. When this is finished, COUT statements are inserted where the line numbers would be on the paper desk check. The resulting code is:

```
{
    int i = 99; // some value
    int j = 0;
    cout << "\ti\tj\n"; // row header of Desk Check table

    cout << "1\t" << i << "\t" << j << endl; // (1)

    for (i = 1; i < 10; i *= 2)
    {
        cout << "2\t" << i << "\t" << j << endl; // (2)
        j += i;
        cout << "3\t" << i << "\t" << j << endl; // (3)
    }

    cout << "4\t" << i << "\t" << j << endl; // (4)
}
```

The output of this modified code should appear “very similar” to the desk check performed by hand:

	i	j
1	99	0
2	1	0
3	1	1
2	2	1
3	2	3
2	4	3
3	4	7
2	8	7
3	8	15
4	16	15

Observe how this table is the same as the paper desk check output.

Challenge

As a challenge, perform a paper and online desk check on the `computeTax()` function from Project 1. What kind of bugs would it help you find?

See Also

The complete solution is available at [2-4-deskCheck.cpp](#) or:

```
/home/cs124/examples/2-4-deskCheck.cpp
```

Pitfalls

As with the pitfalls associated with IF statements, a few pitfalls are common among loops.

Pitfall: = instead of ==

Remember that '=' means assign, and '==' means compare. We almost always want '==' in loops:

```
{
    bool done = false;

    do
    {
        ...
        if (x == 0)
            done = true;
    }
    while (done = false); // PITFALL! We probably want to compare done with false!
}
```

Pitfall: < instead of <=

Pay special attention to the problem you are trying to solve. Some loops require us to add “the numbers less than 100.” This implies `count < 100`. Other loops require us to count “from 1 to 10.” This implies `count <= 10`. This class of errors is called “off-by-one” errors:

```
{
    // count from 1 to 10
    for (int count = 1;
        count < 10;    // PITFALL! The comment says 1 to 10 implying count <= 10
        count++)
    ;
}
```

Pitfall: Extra semicolon

The entire loop statement includes both the loop itself and the body. This means we do not put a semicolon on the FOR loop itself. If we do so, we are implying that there is no body of the loop (just like an IF statement):

```
{
    // count from 1 to 10
    for (int i = 1; i <= 10; i++); // PITFALL! This signifies that there is no body
    cout << i << endl;           // so this statement isn't part of the loop
}
```

Pitfall: Infinite loop

Please make sure that your loop will end eventually:

```
{
    // count from 1 to 10
    for (int i = 1; i > 0; i++)    // PITFALL! I will always be greater than 0!
        cout << i << endl;
}
```

Problem 1

Which of the following is the definition of the conceptual level of understanding of an algorithm?

- What the program does, not how the solution is achieved
- What the components do and how they influence the program
- The value of every variable at every stage of execution
- Realization where the flaws or bugs are

Please see page 168 for a hint.

Problem 2

Where do you put the line numbers in a desk check table?

Answer:

Please see page 171 for a hint.

Problem 3

Given a program that converts feet to meters, create a desk check table for the input value of 2 feet.

```
convert
  PROMPT for feet
  GET feet
  SET meters = feet * 0.301
  PUT meters
END
```


Please see page 171 for a hint.

Problem 4

Desk check the following program.

```
addNumbers
  SET number = 1
  DOWHILE number ≤ 5
    SET number = number + number
  ENDDO
END
```


Please see page 171 for a hint.

Problem 5

What is the output?

```
{
    int i;
    for (i = 0; i < 4; i++)
        ;
    cout << "i == " << i;
}
```

Answer:

Please see page 161 for a hint.

Problem 6

What is the output?

```
{
    bool done = false;
    int n = 5;

    while (!done)
    {
        if (n == 2)
            done = true;
        n--;
    }
    cout << "n == " << n << endl;
}
```

Answer:

Please see page 157 for a hint.

Problem 7

What is the output for the input of 'a' and 'x'?

```
{
    char input;

    do
    {
        cout << "input: ";
        cin >> input;

        cout << "\t"
              << input
              << endl;
    }
    while (input != 'x');
}
```

Answer:

Please see page 157 for a hint.

Problem 8

What is the output?

```
{
    int i;

    for (i = 0; i < 4; i++);
        cout << "H";
    cout << endl;
}
```

Answer:

Please see page 173 for a hint.

Problem 9

What is the output?

```
{
    int i;

    for (i = 0; i <= 4; i++)
        ;

    cout << "i == " << i << endl;
}
```

Answer:

Please see page 157 for a hint.

Problem 10

What is the output?

```
{
    int sum = 0;
    int count;

    for (count = 0;
        count < 4;
        count++)
        sum += count;

    cout << "sum == " << sum;
}
```

Answer:

Please see page 161 for a hint.

Problem 11

What is the output?

```
{
    bool done    = false;
    int  number = 1;

    while (!done)
    {
        cout << number << endl;
        number *= 2;

        if (number > 4)
            done = true;
    }
}
```

Answer:

Please see page 171 for a hint.

Problem 12

What is the output?

```
{
    int sum = 0;
    int count;

    for ( count = 1;
          count < 9;
          count *= 2)
        sum += count;

    cout << "sum == " << sum;
}
```

Answer:

Please see page 171 for a hint.

Problem 13

What is the output?

```
{
    int count = 0;

    while (count < 5)
        count++;

    cout << "count == " << count;
}
```

Answer:

Please see page 157 for a hint.

Problem 14

What is the output?

```
{  
    int count = 10;  
  
    while (count < 5)  
        count++;  
  
    cout << "count == " << count;  
}
```

Answer:

Please see page 157 for a hint.

Problem 15

What is the output?

```
{  
    int count = 0;  
  
    do  
        count++;  
    while (count < 3);  
  
    cout << "count == " << count;  
}
```

Answer:

Please see page 159 for a hint.

Unit 2

Problem 1: Convert grade

A full page of blank graph paper with a uniform grid of small squares. The grid consists of 20 columns and 20 rows, creating a total of 400 squares. The lines are thin and black, set against a white background. There are no margins, text, or other markings on the page.

The user input is 2, 0, 10 in the following code:

This is a full-page image of a blank sheet of white graph paper. The grid consists of thin, light gray horizontal and vertical lines intersecting at right angles to form a uniform pattern of small squares across the entire page. There are no margins, text, or other markings present.

Procedural Programming in C++	Unit 2: Design & Loops	2.4 Loop Output	Page 179
-------------------------------	------------------------	-----------------	-----------------

[illegible]

Unit 2